JAVA PROJECT | OOP | CONSOLE APPLICATION

ChatRoom Application

Name:Shubham Adhikari
Class:2B
Erp:Ru-25-11406
Submitted To: Mr.Shantanu Kumar Sasmal

A console-based multi-user messaging system demonstrating Java OOP principles

Java OOP | 3 Classes | Console App | Multi-User

SLIDE 01

Introduction

The ChatRoom Application is a Java-based console program that simulates a real-time group messaging environment. It demonstrates core Object-Oriented Programming concepts through a clean, modular 3-class design.

Language:	Java (Core Java, No Libraries)
Paradigm:	Object-Oriented Programming
Interface:	Console / Terminal
Storage:	In-memory Array (Message[100])
Purpose:	Simulate multi-user group chat

OOP CONCEPTS USED

Classes & Objects

User, Message, ChatRoom classes

Encapsulation

username with getusername()

Constructors

All 3 classes use constructors

Arrays

Message[100] for storage

Methods

sendMessage(), DisplayMessage()

How It Works

1 Create Users

Instantiate User objects (Alice, Bob, Charlie)

2 Init ChatRoom

ChatRoom created with Message[100] array & messagecount=0

3 Send Message

sendMessage() builds a new Message object and stores it

4 Store Data

Message holds sender (User), content & timestamp

5 Display All

DisplayMessage() loops and prints all messages in order

Class Architecture

class User

- username: String

+ getusername(): String

class Message

- sender: User
- content: String
- timestamp: String
+ Message(User, String, String)

class ChatRoom

- message[]: Message
- messagecount: int
+ sendMessage()
+ DisplayMessage()

SLIDE 03

Source Code — Part 1

User class & Message class

User.java (embedded in main.java)

main.java > ChatRoom > message

```java
import java.util.*;
class User {
    String username;
    User(String username)
    {
        this.username = username;
    }
    public String getusername()
    {
        return username;
    }
}
```

Message.java (embedded in main.java)

```java
class Message{
    User sender;
    String content,timestamp;
    Message( User sender,String content , String timestamp)
    {
        this.sender = sender;
        this.content = content;
        this.timestamp = timestamp;
    }
}
```

SLIDE 04

Source Code — Part 2

ChatRoom class — the core engine

```
class ChatRoom{
    Message [] message;
    ChatRoom()
    {
        message = new Message[100];
        messagecount = 0;
    }
    void sendMessage(User user,String messageText,String time){
        Message msg = new Message(user,messageText,time);
        message [messagecount++] = msg;
    }
    void DisplayMessage()
    {
        for( i =0; i < messagecount; i++)
        {
            Message msg1 = message[i];
            System.out.println("["+ msg1.timestamp+ "] "+msg1.sender.getusername()+": "+msg1.content);
        }
}
}
```

SLIDE 05

Source Code — Part 3

main class — program entry point

main.java — public class main

```java
public class main{
    Run | Debug
    public static void main(String[] args)
    {
        User user1 = new User ("Alice");
        User user2 = new User ("Bob");
        User user3 = new User("Charlie");

        ChatRoom chat = new ChatRoom();
        chat.sendMessage(user1,"Hello everyone!","10:00 AM");
        chat.sendMessage(user2,"Hi Alice!","10:01 AM");
        chat.sendMessage(user3,"Good morning!","10:02 AM");

       chat.DisplayMessage();
    }
}
```

PROBLEMS 1 OUTPUT DEBUG CONSO

```
adhikarishubham41gmail.com@Shubhams-
Chat SImulation Application/" && jav
[10:00 AM] Alice: Hello everyone!
[10:01 AM] Bob: Hi Alice!
[10:02 AM] Charlie: Good morning!
adhikarishubham41gmail.com@Shubhams-
```

KEY NOTES

- messagecount auto-increments on each send
- timestamp stored as plain String
- DisplayMessage loops 0 → messagecount-1
- Sender username fetched via getusername()

Conclusion & Future Scope

CONCLUSION

- Successfully simulates Java OOP with real-world messaging
- 3 well-structured classes with clear separation of concerns
- Array-based storage enables sequential message retrieval
- getusername() demonstrates encapsulation principles

FUTURE SCOPE

Network Sockets
Real TCP/IP messaging

GUI Interface
JavaFX / Swing UI

Authentication
Login & passwords

Database
JDBC message storage